Scaling to 800 Scenes — Case 2: Retraining UNet_Day

Introduction

Case 1 is locked in. The night model converged stably across 40 epochs:

- **Training & Validation Loss:** Dropped from 0.998 down to 0.781.
- **Validation Recall:** Stabilized between 53% and 56% across 100 unseen nighttime scenes.
- **Checkpoint:** Saved to `checkpoints/best_unet_night_800.pth`.

Now we move strictly to **CASE 2: Retraining the Day Model (UNet_Day)** on the new 800-dataset.

Case 2 Mechanics: UNet_Day

- **4-Channel Architecture:** Ingests $T_{3.9}$, $T_{10.8}$, ΔT, plus Channel 3: the 11×11 Local Background Contrast Channel ($C = \Delta T - \text{LocalMean}_{11\times11}$).
- **Symmetric Loss ($\alpha = 0.5$, $\beta = 0.5$):** Uses BalancedDayFireLoss so background non-fire pixels cannot trigger a Dead-Zero collapse, while heavily penalizing daytime solar glint false alarms.
- **Data Filtering:** Isolates daytime overpasses (07:00 $\leq$ UTC Hour $\leq$ 17:00) from `dataset_index_800.csv`.
- **Checkpoint Isolation:** Saves strictly to `checkpoints/best_unet_day_800.pth`.

Step 1: Create `train_day_800.py`

Create this script in `D:\ramy uni\AI ENGENEERING\FINDFIRE\findfire\`:

```
import os
import time
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
from torch.optim import AdamW
from torch.optim.lr_scheduler import CosineAnnealingLR
from torch.utils.data import DataLoader, Dataset

from unet_model import UNet

#
    ======================================================================

# 1. DAY DATASET LOADER (4 CHANNELS WITH 11x11 LOCAL CONTRAST)
#
    ======================================================================

class WildfireDayDataset800(Dataset):
  def __init__(self, index_csv_path, split="train", augment=False
      ):
    self.split = split
    self.augment = augment

    df = pd.read_csv(index_csv_path)
    df["hour"] = pd.to_datetime(df["timestamp"]).dt.hour

    # Filter strictly for daytime scenes (07:00 to 17:00 UTC)
    day_mask = (df["hour"] >= 7) & (df["hour"] <= 17)
    split_mask = df["split"] == split
    self.records = df[day_mask & split_mask].reset_index(drop=
        True)

    if len(self.records) == 0:
      raise RuntimeError(f"No daytime scenes found for split='{
          split}'")

    # Daytime normalization statistics (in Kelvin)
    # Ch 0: T_3.9, Ch 1: T_10.8, Ch 2: Delta_T, Ch 3: Contrast
    self.means = torch.tensor([315.0, 300.0, 15.0, 0.0]).view(4,
        1, 1)
    self.stds = torch.tensor([15.0, 12.0, 8.0, 4.0]).view(4, 1,
        1)

  def __len__(self):
    return len(self.records)

  def __getitem__(self, idx):
    row = self.records.iloc[idx]
    data = np.load(row["file_path"])

    image_3ch = torch.from_numpy(data["image"].astype(np.float32)
        )  # [3, 120, 250]
    mask = torch.from_numpy(data["mask"].astype(np.float32)).
        unsqueeze(0)
```

```
    # Compute 11x11 Local Background Contrast on Delta_T (Channel
        2)
    delta_t = image_3ch[2:3, :, :].unsqueeze(0)  # [1, 1, 120,
       250]
    local_bg = F.avg_pool2d(delta_t, kernel_size=11, stride=1,
       padding=5).squeeze(0)
    contrast = image_3ch[2:3, :, :] - local_bg  # [1, 120, 250]

    # Assemble 4-channel tensor
    image_4ch = torch.cat([image_3ch, contrast], dim=0)  # [4,
       120, 250]

    # Standardize
    image_4ch = (image_4ch - self.means) / self.stds

    # Pad to (128, 256)
    image_padded = F.pad(image_4ch, (3, 3, 4, 4), mode="reflect")
    mask_padded = F.pad(mask, (3, 3, 4, 4), mode="constant",
       value=0.0)

    if self.augment:
      if torch.rand(1).item() > 0.5:
        image_padded = torch.flip(image_padded, dims=[2])
        mask_padded = torch.flip(mask_padded, dims=[2])
      if torch.rand(1).item() > 0.5:
        image_padded = torch.flip(image_padded, dims=[1])
        mask_padded = torch.flip(mask_padded, dims=[1])

    return image_padded, mask_padded


#
   ================================================================

# 2. BALANCED DAYTIME FIRE LOSS (ANTI-COLLAPSE)
#
   ================================================================

class BalancedDayFireLoss(nn.Module):
  def __init__(self, alpha=0.5, beta=0.5, gamma=4.0 / 3.0, eps=1e
     -6):
    super(BalancedDayFireLoss, self).__init__()
    self.alpha = alpha
    self.beta = beta
    self.gamma = gamma
    self.eps = eps

  def forward(self, logits, targets):
    probs = torch.sigmoid(logits)
    p_flat = probs.view(probs.size(0), -1)
    t_flat = targets.view(targets.size(0), -1)
```

```
    tp = torch.sum(p_flat * t_flat, dim=1)
    fp = torch.sum(p_flat * (1.0 - t_flat), dim=1)
    fn = torch.sum((1.0 - p_flat) * t_flat, dim=1)

    ti = (tp + self.eps) / (tp + self.alpha * fp + self.beta * fn
        + self.eps)
    loss = torch.pow((1.0 - ti), (1.0 / self.gamma))
    return torch.mean(loss)


#
   ============================================================================

# 3. TRAINING SETUP & EXECUTION
#
   ============================================================================

INDEX_PATH = r"D:\FINDFIRE_DATA\processed_800\dataset_index_800.
   csv"
CHECKPOINT_DIR = "checkpoints"
os.makedirs(CHECKPOINT_DIR, exist_ok=True)

BATCH_SIZE = 8
EPOCHS = 40
LEARNING_RATE = 2e-4
WEIGHT_DECAY = 1e-4

device = torch.device("cuda" if torch.cuda.is_available() else "
   cpu")
print("====================================================")
print(f"TRAINING UNET_DAY (800 DATASET) ON: {device}")
if device.type == "cuda":
  print(f"GPU: {torch.cuda.get_device_name(0)}")
print("====================================================")

train_dataset = WildfireDayDataset800(INDEX_PATH, split="train",
   augment=True)
val_dataset = WildfireDayDataset800(INDEX_PATH, split="test",
   augment=False)

train_loader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE, shuffle=True,
       num_workers=0
)
val_loader = DataLoader(
    val_dataset, batch_size=BATCH_SIZE, shuffle=False,
       num_workers=0
)

print(f"Daytime Training Scenes   : {len(train_dataset)} ({len(
```

```
    train_loader)} batches)")
print(f"Daytime Validation Scenes : {len(val_dataset)} ({len(
    val_loader)} batches)\n")

# Initialize 4-channel U-Net
model = UNet(in_channels=4, out_channels=1, base_features=64).to(
    device)
criterion = BalancedDayFireLoss(alpha=0.5, beta=0.5, gamma=4.0 /
    3.0)
optimizer = AdamW(model.parameters(), lr=LEARNING_RATE,
    weight_decay=WEIGHT_DECAY)
scheduler = CosineAnnealingLR(optimizer, T_max=EPOCHS, eta_min=1e
    -6)

best_val_dice = 0.0
best_model_path = os.path.join(CHECKPOINT_DIR, "best_unet_day_800
    .pth")

print("Beginning Case 2 Training (UNet_Day)...")
start_time = time.time()

for epoch in range(1, EPOCHS + 1):
  epoch_start = time.time()
  model.train()
  train_loss = 0.0

  for images, masks in train_loader:
    images = images.to(device)
    masks = masks.to(device)

    logits = model(images)
    loss = criterion(logits, masks)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    train_loss += loss.item()

  train_epoch_loss = train_loss / len(train_loader)

  # Validation evaluation
  model.eval()
  val_loss = 0.0
  val_dice_list, val_recall_list = [], []

  with torch.no_grad():
    for images, masks in val_loader:
      images = images.to(device)
      masks = masks.to(device)
```

```
            logits = model(images)
            loss = criterion(logits, masks)
            val_loss += loss.item()

            probs = torch.sigmoid(logits)
            preds = (probs >= 0.50).float()

            p_flat = preds.view(-1)
            t_flat = masks.view(-1)
            tp = torch.sum(p_flat * t_flat).item()
            fp = torch.sum(p_flat * (1.0 - t_flat)).item()
            fn = torch.sum((1.0 - p_flat) * t_flat).item()

            dice = (2.0 * tp + 1e-6) / (2.0 * tp + fp + fn + 1e-6)
            recall = (tp + 1e-6) / (tp + fn + 1e-6)
            val_dice_list.append(dice)
            val_recall_list.append(recall)

    val_epoch_loss = val_loss / len(val_loader)
    mean_val_dice = np.mean(val_dice_list)
    mean_val_recall = np.mean(val_recall_list)

    scheduler.step()
    lr = scheduler.get_last_lr()[0]
    duration = time.time() - epoch_start

    saved_flag = ""
    if mean_val_dice > best_val_dice:
        best_val_dice = mean_val_dice
        torch.save(
            {
                "epoch": epoch,
                "model_state_dict": model.state_dict(),
                "best_dice": best_val_dice,
                "val_loss": val_epoch_loss,
            },
            best_model_path,
        )
        saved_flag = "--> [SAVED BEST DAY MODEL]"

    print(
        f"Epoch [{epoch:02d}/{EPOCHS:02d}] ({duration:.1f}s) | LR:
            {lr:.6f} | "
        f"Train Loss: {train_epoch_loss:.4f} | Val Loss: {
            val_epoch_loss:.4f} | "
        f"Val Dice: {mean_val_dice:.4f} | Val Recall: {
            mean_val_recall:.4f} {saved_flag}"
    )

total_mins = (time.time() - start_time) / 60
print("\n==================================================")
```

```
print("CASE 2 COMPLETE: UNet_Day Training Finished")
print(f"Elapsed Time     : {total_mins:.2f} minutes")
print(f"Best Val Dice    : {best_val_dice:.4f}")
print(f"Saved Checkpoint : {best_model_path}")
print("================================================")
```

Step 2: Run Case 2

Run the daytime training script in your terminal:

```
python train_day_800.py
```

Let it run through its 40 epochs. Once it finishes, share the final log lines so we can move straight to Case 3: Benchmarking and validating the unified pipeline on the 170 test scenes.